

# Insolare Safety System

## Technical Overview

This document explains how the Insolare Safety System works: employee face recognition, PPE kit detection, live camera monitoring, image analysis, and backend data storage.

*Generated from project documentation*

## 1. Overview

The Insolare Safety System is a three-tier safety monitoring application that answers two questions at once:

- Who is this person? (face recognition)
- Are they wearing the required PPE kit? (helmet, gloves, boots, jacket/vest)

The system combines a React frontend, a Node.js backend, and a Python Flask ML server to monitor construction-site safety in real time.

## 2. System Architecture

### Three Main Components

| Layer                        | Role                                                                   |
|------------------------------|------------------------------------------------------------------------|
| Frontend (React, port 5173)  | UI for live camera, image upload, employee admin, dashboard            |
| Backend (Node.js, port 3000) | Auth, employee CRUD, stores detection events and PPE stats in Supabase |
| Flask Server (port 5000)     | All computer vision: face recognition, PPE detection, anti-spoofing    |

### Data Flow

- Frontend Camera Monitor connects to Flask MJPEG stream at /video\_feed
- Image Analysis uploads photos to Flask /api/analyze-image
- Employee photos are saved by the backend and embeddings are generated via createEmbeddings.py
- Flask sends PPE and detection events to the backend API
- Backend stores records in Supabase and can trigger email/SMS alerts

## 3. Employee Enrollment (Face Recognition Setup)

When an admin adds an employee with profile photos:

- Photos are saved to backend/uploads/{employee\_name}/
- Backend runs createEmbeddings.py on that folder
- MTCNN detects the face in each photo
  - FaceNet (InceptionResnetV1, pretrained on VGGFace2) converts each face into a 512-dimensional embedding
- Embeddings are saved as embeddings.csv in the employee folder

At Flask server startup, videoServer.py loads all employee embeddings and builds a Nearest Neighbors index for fast matching during live detection.

## 4. Face Recognition (Who Is This?)

On each processed frame or uploaded image:

- MTCNN detects faces in the image

- Each face is cropped and resized to 160x160 pixels
- FaceNet generates a numeric embedding (face fingerprint)
- The embedding is compared to stored employee embeddings using cosine similarity
- If similarity  $\geq 0.65$ , a match is found; otherwise the person is labeled Unknown

## Anti-Spoofing

Each detected face is also checked using MiniFASNet (AntiSpoofPredict). If a photo or screen is detected instead of a live face, the label becomes "Spoof Detected" and the person is not treated as a valid recognition.

## 5. PPE Kit Detection

PPE detection uses a YOLO object detection model. The system first tries to load a custom trained model from runs/detect/ppe\_detection/weights/best.pt. If that is not available, it falls back to pretrained YOLOv12, YOLOv11, or YOLOv8 weights.

### Required PPE Items

| Item   | Also Detects As                    | Threshold |
|--------|------------------------------------|-----------|
| Helmet | hard-hat, safety-helmet            | 0.75      |
| Gloves | glove, safety-gloves               | 0.65      |
| Boots  | boot, safety-shoes                 | 0.65      |
| Jacket | vest, safety-vest, reflective-vest | 0.65      |

### Detection Pipeline

- YOLO runs inference on the frame with initial confidence  $\geq 0.5$
- Detections are filtered by per-class confidence thresholds
- Spatial validation ensures PPE is near a detected Person bounding box
- Helmets must appear in the upper portion of the person's body
- `check_ppe_compliance()` marks a person compliant only if all 4 items are detected

For live video, spatial rules are strict to reduce false positives. For uploaded images, rules are more lenient because a full person box may not always be detected.

## 6. Live Camera Monitoring

The Camera Monitor page displays a live MJPEG stream from Flask at /video\_feed. The camera can be an RTSP IP security camera or a laptop/webcam.

### Processing Loop

- Every frame is streamed to the browser for live viewing
- Every 8th frame is queued for background ML processing
- Face recognition and PPE detection run in a background thread

- Annotated bounding boxes are drawn on the stream (green = recognized, red = unknown, orange = spoof)
- Flask POSTs to /api/ppe/event once per person per session to update compliance stats
- Flask POSTs to /api/detections/event every 2 seconds per person for Recent Detections
- Snapshot images are saved and shown in the frontend detection history

## 7. Image Analysis (Upload a Photo)

The Image Analysis page uploads images to Flask /api/analyze-image. For images with multiple people, the system:

- Detects all faces in the image
- Runs PPE detection on the whole image
- Matches each PPE item to the nearest person using spatial proximity
- Helmet must be near the face; gloves near hands; boots near feet; jacket near torso

Example response per person:

```
{
  "name": "John Doe",
  "confidence": 0.82,
  "ppe_compliant": false,
  "ppe_items": { "helmet": true, "gloves": false, "boots": true, "jacket": true }
}
```

## 8. Backend Data Storage

When Flask reports a detection, the backend (ppeService.js) updates Supabase:

| Table / Data     | What Is Stored                                               |
|------------------|--------------------------------------------------------------|
| workers          | Daily/total PPE violations, compliance streaks               |
| detection_events | Who was seen, when, PPE status, camera source, snapshot path |
| worker_details   | Employee name to worker ID mapping                           |

The backend can also trigger email and SMS alerts (notificationService.js) when PPE violations are detected, listing which items are missing.

## 9. End-to-End Example

Scenario: Worker Raj walks past the site camera without gloves.

- Camera captures frame; Flask processes it in the background
- MTCNN finds Raj's face; FaceNet embedding matches Raj (confidence 0.78)
- Anti-spoof check passes (real face, not a photo)
- YOLO detects: helmet yes, gloves no, boots yes, jacket yes
- check\_ppe\_compliance() returns non-compliant (gloves missing)
- Flask draws annotations on the live stream

- Flask POSTs to backend; Supabase records the violation
- Frontend Recent Detections panel updates with Raj's snapshot
- Admin can send email/SMS alert from the Camera Monitor UI

## 10. Key Source Files

| File                                 | Purpose                                       |
|--------------------------------------|-----------------------------------------------|
| flaskServer/videoServer.py           | Main ML server: camera stream, image analysis |
| flaskServer/ppeDetection.py          | YOLO PPE detection logic                      |
| flaskServer/createEmbeddings.py      | Generates face embeddings for new employees   |
| backend/app.js                       | API routes, employee management               |
| backend/ppeService.js                | PPE stats and violation tracking              |
| frontend/src/Pages/CameraMonitor.jsx | Live camera UI                                |
| frontend/src/Pages/ImageAnalysis.jsx | Upload and analyze images                     |

## 11. Summary

| Capability      | Technology                  | How It Works                                             |
|-----------------|-----------------------------|----------------------------------------------------------|
| Who             | MTCNN + FaceNet             | Face to embedding to nearest match vs enrolled employees |
| Spoof check     | MiniFASNet                  | Rejects photos/screens of faces                          |
| PPE kit         | YOLO (custom or pretrained) | Detects helmet, gloves, boots, jacket with spatial rules |
| Live monitoring | Flask MJPEG + threads       | Processes every 8th frame, streams annotated video       |
| Records         | Node.js + Supabase          | Stores detections, violations, sends alerts              |